

ChurchManager Permanent Hymn Identifier Specification

Status: Proposed; awaiting approval

Prepared: August 15, 2026

Scope: Development ChurchManager catalog design, installer packages, and ChurchDBTest

Out of scope: Frozen ChurchManager-Legacy and production ChurchDB

1. Purpose

Give every packaged and locally entered hymn a permanent numeric HymnID that has the same meaning for the lifetime of that record. Installing hymnals in a different order must never change, translate, or accidentally redirect hymn references.

MariaDB primary and foreign keys do not need to be consecutive. Gaps between permanent ID blocks have no meaningful storage or performance penalty.

2. Core decision

`tblHymn`.ID remains a signed 32-bit INTEGER, but it will not use `AUTO_INCREMENT`.

Every hymnal receives a permanent 5,000-number block:

```
HymnID = HymnalID * 5,000 + EntrySlot
```

`HymnalID` is itself a permanent registered catalog number. `EntrySlot` is from 1 through 4,999. Slot zero is reserved and is never assigned to a hymn.

The `tblHymnal` record stores its assignment explicitly:

```
HymnIDStart integer NOT NULL,  
HymnIDEnd integer NOT NULL
```

For a standard block, `HymnIDStart` is `HymnalID * 5,000 + 1` and `HymnIDEnd` is `HymnalID * 5,000 + 4,999`. The stored range is authoritative for installer and database validation. Code may calculate the expected range as a consistency check, but it must not ignore the range recorded on the hymnal.

This permits more than 400,000 catalog blocks within a signed 32-bit integer, far beyond ChurchManager's foreseeable need.

3. Initial block registry

HymnalID	HymnID range	Assignment
0	1-4,999	Reserved; never distributed
1	5,001-9,999	Reserved for future system use
2	10,001-14,999	Local user-entered hymns
3	15,001-19,999	LSB - Lutheran Service Book
4 and above	successive 5,000-number blocks	Assigned permanently through the maintained registry

Values 0, 5,000, 10,000, 15,000, and every later block boundary are reserved slot-zero values. NULL, not zero, represents "No hymnal" where that is valid.

The LSB assignment becomes permanent once the migration is approved. Other hymnals receive a block only when their package is approved; speculative block assignments are avoided.

4. Entry-slot assignment

For a straightforward numbered hymn, the printed hymn number should normally be its entry slot. Under this rule, LSB 656 receives:

```
HymnalID = 3  
EntrySlot = 656  
HymnID = 15,656
```

The printed reference remains a separate database value. It is not calculated from HymnID at runtime.

An alternate unused slot must be assigned when a catalog contains:

- duplicate printed numbers;
- lettered or otherwise non-integer numbers;
- unnumbered entries;
- authorized supplementary material; or
- another entry that cannot safely use its printed number.

Those assignments become part of the permanent package manifest. They are never derived differently by individual installations.

5. User-entered hymns

All local user-entered hymns belong to the reserved local catalog:

```
HymnalID = 2  
HymnID range = 10,001 through 14,999
```

ChurchManager assigns the next never-used value in this range. The ID becomes permanent immediately.

- Local IDs are never placed in a distributed starter package.
- A locally entered hymn may be used normally in worship history and local suggestions.
- Removing a local hymn retires it; its ID is never reused.
- The user is warned well before the 4,999-entry capacity is reached.

Five thousand local entries are considered sufficient for one congregation. Expanding the local range would require a new approved registry block, not reuse of another catalog's range.

6. Database relationships

The following columns must use the same signed INTEGER type:

- `tblHymn.ID` - permanent parent key
- `tblHymnUsage.HymnID` - worship-history reference
- `tblProperHymnSuggestion.HymnID` - Proper suggestion reference

Any future HymnID foreign key must follow this specification.

`tblHymn.HymnaLID` references the permanent `tblHymnaL.ID`. The database must enforce that every hymn falls inside the block assigned to its `HymnaLID`. Application validation and package tests provide the primary enforcement; a database check or trigger may be added if it can be implemented clearly and portably.

`tblHymnaL.HymnIDStart` and `tblHymnaL.HymnIDEnd` must be positive, ordered, non-overlapping, consistent with the permanent registry, and immutable after a hymn has been assigned in the range.

7. Deletion and historical integrity

Packaged and used hymns are never physically deleted during ordinary operation. Add `IsActive` to `tblHymn` and retire records instead.

The existing cascading relationship from `tblHymnUsage.HymnID` must be replaced with `ON DELETE RESTRICT`. Deleting a hymn must never delete historical worship usage. The Proper-suggestion relationship may remove a suggestion explicitly, but it must not cause worship history to disappear.

An inactive hymn:

- remains visible in historical services and reports;
- is excluded from ordinary new-hymn selection unless an administrator asks to include inactive entries; and
- may be reactivated without changing its ID.

8. Package format

Every hymnal package contains:

- permanent `HymnaLID`;
- permanent `HymnIDStart` and `HymnIDEnd`;
- package code, title, version, source, and license information;
- its assigned 5,000-number range;
- an explicit permanent `HymnID` for every entry;
- printed reference, title, tune, category, and other approved metadata; and
- checksums for the package and its data files.

Proper suggestions and Order of Service starter packages reference the permanent numeric `HymnID` directly. They must declare the required hymnal-package version as a dependency.

Textual package codes such as `1sb` remain useful human-readable metadata and a secondary uniqueness check. They are not used to translate hymn foreign keys.

9. Installation and upgrade rules

Catalog installation occurs in one database transaction:

1. Verify package signature or approved checksum, package code, version, and license metadata.
2. Verify that `HymnaLID` owns the declared block.
3. Verify that the package range exactly matches `tblHymnaL.HymnIDStart` and `tblHymnaL.HymnIDEnd`, or insert those approved values for a new hymnal.

4. Verify that every packaged HymnID lies within that range.
5. Reject duplicate IDs within the package.
6. If an ID already exists, verify that it belongs to the same hymnal and entry.
7. Insert or update the hymnal and hymn records.
8. Only then install dependent Proper suggestions and Order of Service starters.
9. Roll back everything if any ID, dependency, or content check fails.

Installation order therefore has no effect on hymn identity.

An upgrade may correct approved metadata or add unused IDs. It may not:

- renumber an entry;
- reuse a retired ID;
- silently replace one hymn with another;
- alter a local user-entered range; or
- physically remove a hymn referenced by history.

10. Migration of current LSB data

The existing test database uses local historical IDs that cannot be assumed to equal printed hymn numbers. Migration requires an explicit old-to-new mapping.

The migration process will:

1. Back up ChurchDBTest.
2. Validate every current LSB entry and its printed reference.
3. Build and preserve an old-ID to permanent-ID conversion log.
4. Reject duplicate or ambiguous printed references for manual review.
5. Assign ordinary printed LSB hymns to slots matching their printed numbers.
6. Assign approved exceptional entries to documented unused slots.
7. Update `tblHymnUsage.HymnID` and `tblProperHymnSuggestion.HymnID` in the same transaction.
8. Assign permanent `tblHymnal.ID` values and update all hymnal foreign keys.
9. Replace cascade deletion with retirement and restrictive foreign keys.
10. Verify that every child reference resolves and row counts are unchanged.

No conversion will be applied to production ChurchDB as part of development.

11. Validation and tests

Automated and migration tests must prove:

- block calculations and boundaries;
- stored hymnal ranges, registry consistency, and overlap detection;
- reserved slot-zero values cannot be assigned;
- official and local ranges cannot overlap;
- local allocation never reuses a retired ID;
- every foreign-key column has compatible type and signedness;
- packages cannot insert outside their assigned block;
- duplicate or conflicting package IDs roll back the transaction;
- installation order does not change IDs;
- upgrades preserve existing IDs;
- used hymns cannot be physically deleted;
- inactive hymns remain readable in history;
- all current LSB usage and Proper suggestions survive migration; and
- old-to-new migration counts and references reconcile exactly.

12. Acceptance criteria

The design is complete when:

- the permanent hymnal-block registry is approved;
- LSB and local-user blocks are fixed permanently;
- all current hymn relationships are included in the migration plan;
- deletion is replaced by retirement;
- installer and upgrade behavior is transactional and collision-safe;
- an explicit current-LSB conversion report has no unresolved entries; and
- ChurchDBTest migration and full worship-planning regression tests pass.